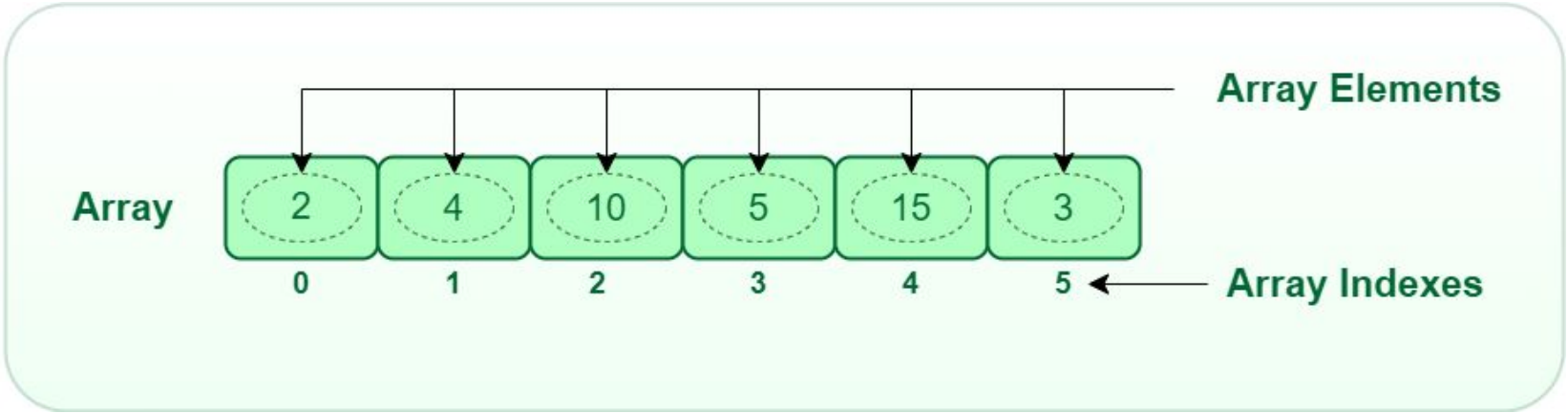
# Data Structures

## Lecture 1
## Array Basics

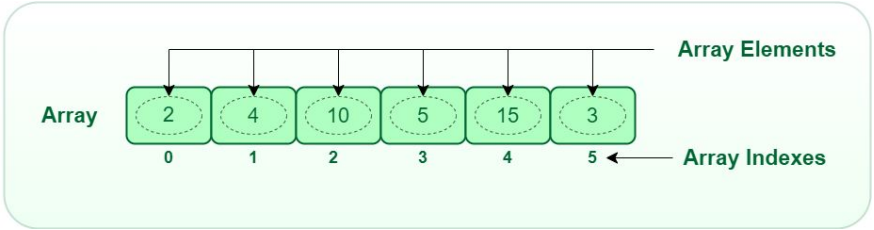
Prantik Paul [PNP]
Lecturer
Department of Computer Science and Engineering
BRAC University

# Array - Basics



# Array - Basics

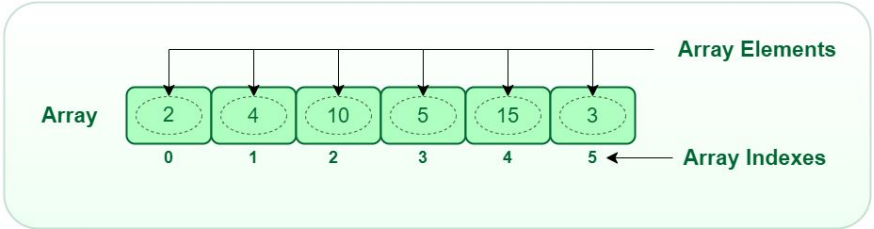
1. Dimension - fixed ( 1, 2, 3, ..., 100, ..)
2. Index
    a. 0 - indexing
    b. 1 - indexing
3. Size / Length
4. Type - fixed, can’t mix



# Array - Basics

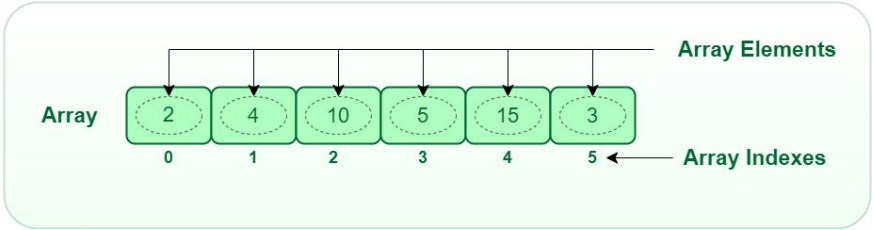
1. Dimension - fixed ( 1, 2, 3, ..., 100, ..)

   Linear Arrays - 1D
   Matrix - 2D
   Tensor - 3D



# Array - Basics

2. Index
    a. 0 - indexing
    b. 1 - indexing




**Why 0 indexing?**

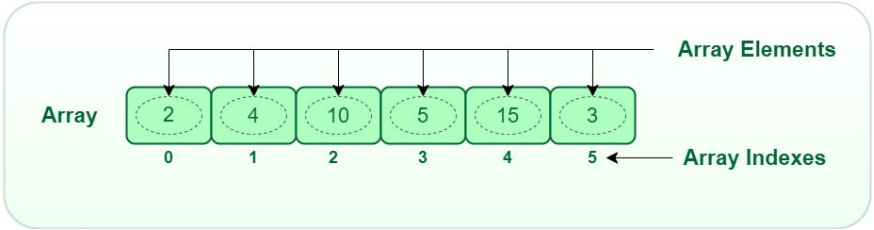
For efficient memory access

# Array - Basics

3. Size / Length

**Size?**
Number of valid elements

**Length?**
Fixed (as declared)

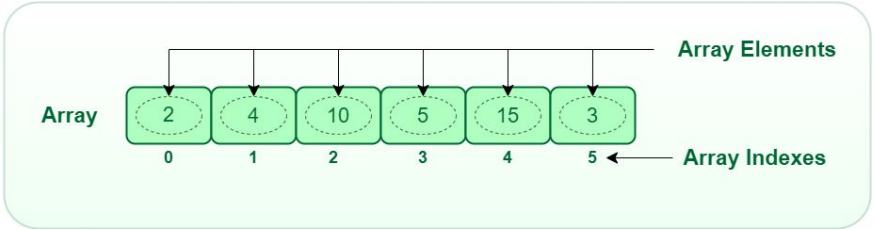


# Array - Basics

4. Type - fixed, can't mix

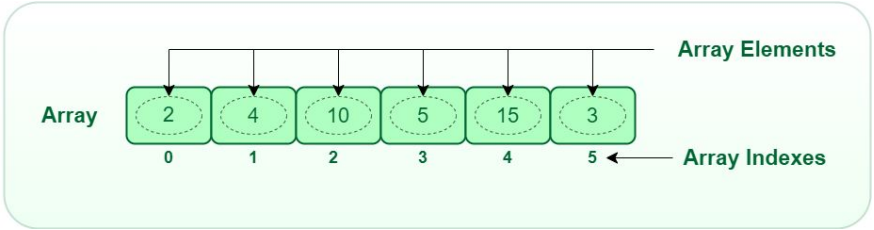
**Why not?**

For efficient memory access



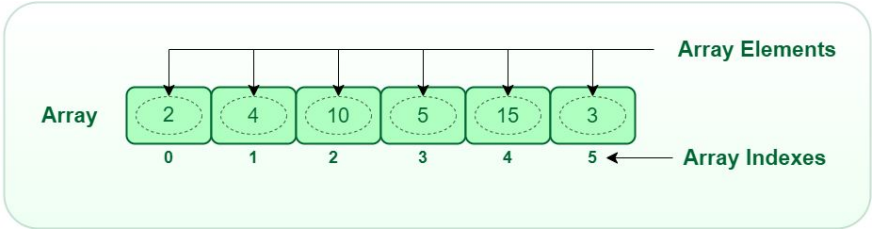
# Array - Accessing an Element

**Formula :**
Starting address of array + index*bytes required for one data




**Integer :** 4bytes
**Floats :** 8bytes
**Character :** 1byte

# Array - Determine Length




**Efficient :**

-> Doesn’t store size

-> May give invalid data if index out of bounds

-> can access faster

-> C Fortran

**Inefficient :**

-> Store size in first byte

-> check if index out of bound

-> slow access

-> Java c#